

Luxury Resort: стек технологій і відповіді на питання для захисту

1. Коротко про саму програму

Чому обрали саме цю тему?

Тему готельно-курортного порталу обрано тому, що вона добре демонструє повний цикл роботи сучасної веб-системи: реєстрація користувача, авторизація, каталог, бронювання, оплата, додаткові послуги, відгуки, адміністрування та аналітика.

Це не просто статичний сайт, а приклад інформаційної системи з ролями, базою даних, бізнес-логікою і взаємодією клієнтської та серверної частин.

Яку проблему вирішує система?

Система вирішує проблему ручного та розрізненого керування бронюваннями, послугами й відгуками в готелі. Замість того щоб вести все через телефон, таблиці або паперові записи, портал дає єдине місце для:

- перегляду номерів;
- онлайн-бронювання;
- замовлення послуг;
- контролю статусів;
- отримання чеків;
- модерації відгуків;
- перегляду аналітики.

Хто є користувачами програми?

Основні користувачі:

- гість готелю;
- працівник рецепції;
- менеджер;
- адміністратор.

Гість бронює та замовляє. Ресепшн працює із замовленнями послуг. Менеджер керує контентом і модерує. Адміністратор має повний доступ до системи.

Які основні функції реалізовані?

Основні функції:

- реєстрація та вхід;
- JWT-авторизація;

- швидкий демо-вхід;
- каталог номерів;
- каталог послуг;
- фільтрація, сортування, пагінація;
- бронювання номерів;
- перевірка доступності дат;
- динамічне ціноутворення;
- демо-оплата;
- PDF-чек;
- замовлення послуг;
- особистий кабінет;
- профіль користувача;
- відгуки та модерація;
- адмін-панель;
- керування номерами, послугами, користувачами;
- аналітика;
- аудит дій.

Які переваги рішення перед аналогами?

Переваги:

- єдиний портал для номерів, послуг і відгуків;
- зручний особистий кабінет;
- розмежування ролей;
- український інтерфейс;
- демо-оплата та PDF-чек;
- аналітика для адміністрації;
- Docker-запуск;
- зрозуміла архітектура front-end + back-end + database;
- можливість демонструвати повний бізнес-процес від вибору номера до оплати.

2. Архітектура

Яка структура програми?

Система побудована за клієнт-серверною архітектурою.

Основні частини:

- Front-end: React-додаток, який працює в браузері.
- Back-end: Spring Boot API, який обробляє запити та бізнес-логіку.

- Database: PostgreSQL, де зберігаються користувачі, номери, бронювання, послуги, платежі та інші дані.
- Docker Compose: запускає базу, сервер і клієнтську частину як окремі контейнери.

У front-end є сторінки, компоненти, хуки, API-клієнти, типи та допоміжні функції.

У back-end є контролери, сервіси, репозиторії, DTO, mapper-и, entity-класи, security-конфігурація та Flyway-міграції.

Чому обрали саме таку архітектуру?

Клієнт-серверна архітектура добре підходить для веб-порталу, бо:

- інтерфейс відокремлений від бізнес-логіки;
- back-end можна використовувати не лише для веб-сайту, а й для мобільного застосунку;
- база даних ізольована від прямого доступу користувача;
- простіше масштабувати частини системи окремо;
- простіше тестувати front-end і back-end незалежно.

Для дипломного проєкту це також демонструє сучасний підхід до розробки повноцінних веб-систем.

Як взаємодіють клієнтська та серверна частини?

Клієнтська частина надсилає HTTP-запити до REST API.

Наприклад:

- користувач натискає «Увійти»;
- React надсилає POST-запит на сервер;
- сервер перевіряє email і пароль;
- сервер повертає JWT-токени й дані користувача;
- клієнт зберігає токен і використовує його в наступних запитах.

Для захищених операцій клієнт додає access token у заголовок Authorization. Сервер перевіряє токен і визначає, хто виконує дію.

Які модулі є в системі?

Основні back-end модулі:

- auth — реєстрація, логін, refresh token, профіль;
- rooms — каталог номерів і доступність;
- bookings — створення, перегляд, статуси бронювань;
- payments — демо-оплата бронювання;
- services — каталог послуг;
- service-orders — замовлення послуг;
- reviews — відгуки та модерація;
- admin — керування системою;

- analytics — статистика й графіки;
- pricing — правила динамічного ціноутворення;
- audit — журнал дій;
- invoice — генерація PDF-чеків.

Основні front-end модулі:

- pages — сторінки застосунку;
- components — повторно використовувані UI-блоки;
- api — функції для запитів до back-end;
- hooks — React Query hooks;
- store — стан авторизації;
- types — TypeScript-типи;
- lib — форматування, labels, validation, допоміжні функції.

3. База даних

Яку СУБД використовували?

Використано PostgreSQL.

Чому саме PostgreSQL?

PostgreSQL обрано тому, що це надійна реляційна СУБД з відкритим кодом. Вона підтримує:

- транзакції;
- зовнішні ключі;
- індекси;
- enum-типи;
- JSONB;
- складні обмеження;
- стабільну роботу з Spring Data JPA.

Для системи бронювань важливо мати цілісність даних. Наприклад, бронювання має бути пов'язане з існуючим користувачем і номером, а платежі — з бронюваннями.

Які таблиці створені?

Основні таблиці:

- users — користувачі;
- rooms — категорії номерів;
- bookings — бронювання;
- payments — платежі;
- services — послуги;

- `service_orders` — замовлення послуг;
- `reviews` — відгуки;
- `pricing_rules` — правила ціноутворення;
- `audit_logs` — журнал дій;
- `refresh_tokens` — токени оновлення.

Які зв'язки між таблицями?

Основні зв'язки:

- `users 1:N bookings` — один користувач може мати багато бронювань;
- `rooms 1:N bookings` — один номер/категорія може мати багато бронювань у різні дати;
- `bookings 1:N payments` — бронювання може мати пов'язані платежі;
- `services 1:N service_orders` — одна послуга може бути замовлена багато разів;
- `users 1:N service_orders` — один користувач може мати багато замовлень послуг;
- `bookings 1:N service_orders` — послуга може бути прив'язана до бронювання;
- `bookings 1:1` або `1:N reviews` логічно — відгук створюється по бронюванню;
- `users 1:N reviews` — користувач може залишати відгуки;
- `users 1:N audit_logs` — користувач може бути виконавцем дії в журналі.

Що таке первинний ключ?

Первинний ключ — це поле або набір полів, які унікально ідентифікують запис у таблиці.

У проєкті як первинні ключі використовуються UUID. Наприклад, кожен користувач, номер, бронювання або замовлення має власний UUID.

Перевага UUID у тому, що він унікальний і не залежить від порядкового номера в таблиці.

Що таке зовнішній ключ?

Зовнішній ключ — це поле, яке посилається на первинний ключ іншої таблиці.

Наприклад:

- `bookings.user_id` посилається на `users.id`;
- `bookings.room_id` посилається на `rooms.id`;
- `payments.booking_id` посилається на `bookings.id`;
- `service_orders.service_id` посилається на `services.id`.

Зовнішні ключі не дозволяють створити «відірвані» записи. Наприклад, не можна створити платіж для бронювання, якого не існує.

Що таке нормалізація бази даних?

Нормалізація — це організація таблиць так, щоб уникнути дублювання даних і зберегти логічні зв'язки.

Наприклад, дані користувача не дублюються в кожному бронюванні. У бронюванні зберігається `user_id`, а повна інформація про користувача лежить у таблиці `users`.

Це зменшує дублювання і полегшує оновлення даних.

4. Програмування

Якою мовою написана програма?

Back-end написаний мовою Java 21.

Front-end написаний мовою TypeScript з React.

SQL використовується для структури бази даних, міграцій і seed-даних.

Чому обрали саме Java?

Java добре підходить для серверної частини, тому що:

- має сильну типізацію;
- стабільна в enterprise-розробці;
- добре підтримується Spring Boot;
- має зручні інструменти для роботи з базою даних;
- підходить для складної бізнес-логіки;
- має велику екосистему бібліотек.

Чому обрали TypeScript і React?

React обрано для створення інтерактивного SPA-інтерфейсу.

TypeScript додає типізацію до JavaScript, що зменшує кількість помилок на front-end.

Це корисно, бо дані з API мають конкретну форму: користувачі, бронювання, послуги, платежі, статуси.

Які бібліотеки або фреймворки використовували?

Back-end:

- Spring Boot;
- Spring Web;
- Spring Security;
- Spring Data JPA;
- Hibernate;
- PostgreSQL JDBC Driver;
- Flyway;
- MapStruct;
- Lombok;

- JWT;
- iText PDF;
- Bucket4j;
- Springdoc OpenAPI;
- Testcontainers.

Front-end:

- React;
- TypeScript;
- Vite;
- React Router;
- TanStack Query;
- Axios;
- Zustand;
- React Hook Form;
- Zod;
- Tailwind CSS;
- Radix UI;
- Recharts;
- Framer Motion;
- Lucide React;
- Vitest;
- Testing Library.

DevOps:

- Docker;
- Docker Compose;
- Nginx для роздачі front-end у контейнері;
- PostgreSQL container.

Що таке ООП?

ООП — об'єктно-орієнтоване програмування. Це підхід, у якому програма будується з об'єктів, що поєднують дані та поведінку.

Наприклад, у проєкті є сутності User, Room, Booking, Payment, ServiceOrder. Вони відображають реальні об'єкти предметної області.

Які принципи ООП використовували?

Використані принципи:

- інкапсуляція — дані сутностей приховані за класами та методами;

- абстракція — робота з репозиторіями та сервісами через інтерфейси;
- наслідування — використовується через фреймворки, наприклад базові класи та інтерфейси Spring Data;
- поліморфізм — різні реалізації можуть використовуватись через спільні інтерфейси.

Також застосовано розділення відповідальностей:

- controller приймає HTTP-запит;
- service містить бізнес-логіку;
- repository працює з базою;
- mapper перетворює entity в DTO.

5. Безпека

Як реалізована авторизація?

Авторизація реалізована через JWT.

Після входу сервер повертає:

- access token;
- refresh token;
- дані користувача.

Access token використовується для запитів до захищених endpoint-ів. Refresh token потрібен для оновлення сесії.

Як захищаються паролі?

Паролі не зберігаються у відкритому вигляді.

Вони хешуються за допомогою BCrypt. Це означає, що в базі лежить не пароль, а його криптографічний хеш.

Навіть якщо хтось отримає доступ до бази, він не побачить реальні паролі користувачів.

Як запобігти SQL-ін'єкціям?

У проєкті використовується Spring Data JPA та параметризовані запити.

Це означає, що значення користувача не вставляються напряму в SQL-рядок. Вони передаються як параметри, які обробляє JPA/драйвер бази.

Також застосовуються DTO і validation, що зменшує ризик некоректних даних.

Як перевіряються дані користувача?

Дані перевіряються на двох рівнях:

- front-end: Zod-схеми й валідація форм;
- back-end: Jakarta Validation у DTO.

Наприклад:

- email має бути коректним;
- пароль має мати мінімальну довжину;
- дата виїзду має бути після дати заїзду;
- кількість гостей не може перевищувати місткість номера;
- кількість послуг має бути в допустимих межах.

Як захищається доступ до чужих даних?

Сервер перевіряє користувача з JWT-токена.

Гість може бачити тільки свої бронювання, свої замовлення послуг і свої доступні дії. Навіть якщо вручну змінити URL або id, сервер має перевірити право доступу.

Для адміністративних сторінок використовується перевірка ролей.

6. Тестування

Як тестували програму?

Front-end тестувався через Vitest і Testing Library.

Перевірялись:

- валідація форми бронювання;
- розрахунок і показ ціни;
- процес демо-оплати;
- рекомендації послуг;
- робота допоміжних функцій query-параметрів.

Back-end у проєкті має залежності для Spring Boot Test і Testcontainers, що дозволяє тестувати код із реальною PostgreSQL у контейнері.

Також виконувалась ручна перевірка основних user flow:

- реєстрація;
- логін;
- швидкий демо-вхід;
- бронювання;
- PDF-чек;
- замовлення послуги;
- фільтри й сортування;
- відгуки;
- адмін-панель.

Які помилки були знайдені?

Під час роботи були знайдені й виправлені такі проблеми:

- дублікати послуг у каталозі;
- демо-номери типу «Бізнес номер 7»;
- некоректна підсвітка пункту «Кабінет»;
- недостатнє розмежування навігації сайту й акаунта;
- непрацюючі фільтри через подвійне оновлення URL-параметрів;
- англійські статуси в PDF-чеку;
- відсутній import Query у RoomRepository;
- Flyway checksum mismatch для demo seed-файлів;
- зламані seed-зображення;
- misleading графік прогнозу, який був уточнений як тренд зайнятості, а не прогноз прибутку.

Які сценарії перевіряли?

Перевірені сценарії:

- гість входить через демо-вхід;
- гість переглядає номери;
- гість бронює номер;
- гість оплачує бронювання;
- гість завантажує PDF;
- гість фільтрує бронювання;
- гість замовляє послугу;
- гість фільтрує послуги;
- гість залишає відгук;
- ресепшн переглядає замовлення послуг;
- адміністратор переглядає аналітику;
- адміністратор керує каталогом.

7. Часті теоретичні питання

Що таке API?

API — це інтерфейс, через який одна частина програми взаємодіє з іншою.

У цьому проєкті front-end використовує API back-end сервера. Наприклад, щоб отримати список номерів, клієнт надсилає HTTP-запит до endpoint-а `/api/rooms`.

Що таке REST?

REST — це архітектурний стиль для побудови веб-API.

У REST ресурси представлені URL-адресами, а дії виконуються HTTP-методами:

- GET — отримати;
- POST — створити;
- PUT — оновити;
- DELETE — видалити.

Чим GET відрізняється від POST?

GET використовується для отримання даних і не повинен змінювати стан системи.

POST використовується для створення або виконання дії, яка змінює стан.

Приклад:

- GET /api/rooms — отримати номери;
- POST /api/bookings — створити бронювання.

Що таке Git?

Git — це система контролю версій. Вона дозволяє відстежувати зміни в коді, повертатися до попередніх версій і працювати над проектом командно.

Що таке Docker?

Docker — це інструмент для запуску застосунків у контейнерах.

У цьому проєкті Docker Compose запускає:

- PostgreSQL;
- back-end;
- front-end.

Це спрощує запуск проєкту на іншому комп'ютері, бо не потрібно вручну налаштовувати всі сервіси окремо.

Що таке Front-end і Back-end?

Front-end — це частина, яку бачить користувач у браузері: сторінки, кнопки, форми, картки.

Back-end — це серверна частина: API, бізнес-логіка, робота з базою, авторизація, перевірки.

Що таке хешування?

Хешування — це перетворення даних у рядок фіксованого вигляду. Для паролів це використовується так, щоб не зберігати сам пароль.

У проєкті використовується BCrypt. Він спеціально створений для безпечного хешування паролів.

Що таке JWT?

JWT — це токен, який містить інформацію про користувача і підписується сервером.

Клієнт додає JWT до запитів, а сервер перевіряє підпис і визначає, чи має користувач доступ.

Що таке ORM?

ORM — це технологія, яка дозволяє працювати з таблицями бази даних як з об'єктами мови програмування.

У проєкті використовується JPA/Hibernate. Наприклад, таблиця bookings відповідає Java-класу Booking.

Що таке DTO?

DTO — Data Transfer Object. Це об'єкт для передачі даних між шарами або між сервером і клієнтом.

DTO потрібні, щоб не віддавати напряму entity-класи з бази й контролювати, які саме поля бачить клієнт.

Що таке міграції бази даних?

Міграції — це файли, які послідовно змінюють структуру або дані бази.

У проєкті використовується Flyway. Файли V1, V2, V3 і далі застосовуються в правильному порядку.

Що таке seed-дані?

Seed-дані — це початкові демо-дані, які додаються в базу для тестування й демонстрації.

У цьому проєкті seed містить:

- демо-користувачів;
- номери;
- послуги;
- бронювання;
- платежі;
- відгуки;
- аналітичні дані.

8. Стек проєкту

Front-end stack

- React 18;
- TypeScript;
- Vite;
- React Router;
- TanStack Query;
- Axios;
- Zustand;

- React Hook Form;
- Zod;
- Tailwind CSS;
- Radix UI;
- Recharts;
- Framer Motion;
- Lucide React;
- Vitest;
- Testing Library.

Back-end stack

- Java 21;
- Spring Boot 3.4.2;
- Spring Web;
- Spring Security;
- Spring Data JPA;
- Hibernate;
- Jakarta Validation;
- PostgreSQL Driver;
- Flyway;
- MapStruct;
- Lombok;
- JWT;
- iText PDF;
- Bucket4j;
- Springdoc OpenAPI;
- Spring Boot Actuator.

Database stack

- PostgreSQL 16;
- Flyway migrations;
- UUID primary keys;
- enum types;
- JSONB для деяких snapshot-даних.

Infrastructure stack

- Docker;
- Docker Compose;
- Nginx для front-end контейнера;

- Maven для back-end збірки;
- npm для front-end збірки.

9. Як коротко пояснити архітектуру на захисті

Можна відповісти так:

«Мій проєкт побудований за клієнт-серверною архітектурою. Клієнтська частина написана на React і TypeScript, вона відповідає за інтерфейс користувача. Серверна частина написана на Java з використанням Spring Boot і відповідає за бізнес-логіку, авторизацію, роботу з бронюваннями, послугами, платежами та відгуками. Дані зберігаються в PostgreSQL. Взаємодія між клієнтом і сервером відбувається через REST API. Для запуску використовується Docker Compose, який піднімає базу даних, back-end і front-end.»

10. Як коротко пояснити безпеку на захисті

Можна відповісти так:

«Безпека реалізована через Spring Security і JWT. Після входу користувач отримує токен, який використовується для доступу до захищених endpoint-ів. Паролі зберігаються не у відкритому вигляді, а як BCrypt-хеші. Доступ до даних обмежений ролями: гість бачить тільки свої бронювання й послуги, персонал має окремі права, адміністратор має повний доступ. SQL-ін'єкціям запобігаємо через JPA, параметризовані запити та validation DTO.»

11. Як коротко пояснити тестування на захисті

Можна відповісти так:

«Я тестував програму автоматично й вручну. На front-end використано Vitest і Testing Library для перевірки компонентів, форм і ключових сценаріїв. Також вручну перевірялись user flow: реєстрація, вхід, бронювання, демо-оплата, PDF-чек, замовлення послуг, фільтри та ролі. Для back-end передбачені Spring Boot Test і Testcontainers, що дозволяє тестувати сервер із реальною PostgreSQL у контейнері.»